

Инструкция по запуску и наполнению данными

локальной информационно-поисковой RAG-системы по
русскоязычной базе знаний Python

Документ содержит воспроизводимую последовательность первого
запуска, загрузки локальных моделей, демонстрационного наполнения,
импорта реального корпуса и построения поискового индекса.

Репозиторий	https://github.com/4iriks/praktika
Ветка	main
Git commit	60cc0680cba0934d2359701778e743f51f397a8d
Дата формирования	17 июля 2026 г.
Основание	README.md, Makefile, compose.yaml и фактические CLI-модули проекта

1. Назначение и безопасный сценарий

Все команды выполняются из корня репозитория PyAnswer. Рекомендуемый порядок: инфраструктура и миграции → модели → наполнение PostgreSQL → индексирование в Qdrant → проверки.



Важно. Полный импорт, загрузка моделей и полная переиндексация требуют времени, сети, места на диске и вычислительных ресурсов. Они защищены явными флагами и в ходе подготовки этой инструкции не запускались. Команда `make demo-down` сохраняет volumes; не используйте `docker compose down -v`, если данные нужны.

Что запускается

Сервис	Роль	Доступ с хоста
frontend / Nginx	Единый origin, React UI и проксирование /api	http://localhost:8080
backend	FastAPI, авторизация, поиск, RAG и администрирование	127.0.0.1:8000; через UI — /api
postgres	Пользователи, корпус, jobs, история и аудит	127.0.0.1:5432
qdrant	Sparse BM25 и dense HNSW индекс	127.0.0.1:6333
ollama	Локальные embedding- и generation-модели	только внутренняя Docker-сеть
worker / indexer	Импорт и индексирование через durable jobs	внутренняя Docker-сеть
reranker	Локальное переранжирование кандидатов	внутренняя Docker-сеть

2. Системные требования

- Linux/Ubuntu, Docker Engine и Docker Compose plugin либо совместимый docker-compose;
- рекомендуется 32 ГБ RAM; CPU-режим поддерживается, NVIDIA GPU необязателен;
- для полного локального комплекта проект предусматривает до 35 ГБ дискового бюджета;
- доступ в интернет нужен для Docker images, моделей и реального корпуса.

Проверка перед стартом:

```
docker --version
docker compose version
df -h .
```

Если используется отдельная GPU-конфигурация `compose.gpu.yaml`, требуются рабочий NVIDIA driver и NVIDIA Container Toolkit. Базовая инструкция ниже подходит для CPU.

3. Первый запуск

- 1 **Перейдите в корень проекта.**

```
cd /путь/к/praktika
```

2 Запустите идемпотентную первичную настройку.

```
make first-run
```

Команда проверяет Docker и свободное место, создаёт локальный `.env` только при его отсутствии, поднимает PostgreSQL/Qdrant/Ollama, выполняет `alembic upgrade head`, `bootstrap` и `demo seed`, затем запускает `backend`, `worker`, `indexer`, `reranker` и `frontend`.

3 Проверьте и загрузите локальные модели.

```
make models-check  
make models-pull  
make reranker-model-pull  
make models-check
```

`models-pull` загружает Ollama-модели `qwen3-embedding:0.6b` и `qwen3:8b`. Отдельная команда готовит `Qwen/Qwen3-Reranker-0.6B` в persistent volume `reranker`.

4 Поднимите/актуализируйте весь демо-стек и проверьте контейнеры.

```
make demo-up  
make demo-status
```

5 Откройте приложение.

```
http://localhost:8080  
http://localhost:8080/api/docs
```

Что не делает `make first-run`. Команда не запускает полный импорт реального корпуса, не иницирует полную переиндексацию и не должна автоматически скачивать большие модели. Эти действия выполняются отдельно и осознанно.

Локальные демонстрационные учётные записи

Роль	Email	Пароль после свежего first-run
USER	user@pyanswer.local	Demo123!
EDITOR	editor@pyanswer.local	Demo123!
ADMIN	admin@pyanswer.local	локальное значение BOOTSTRAP_ADMIN_PASSWORD из .env

Почему пароль ADMIN отличается. В фактической последовательности first-run bootstrap создаёт admin@pyanswer.local раньше demo seed. Генератор .env назначает ему случайный пароль; seed существующего пользователя не перезаписывает. Значение является локальным секретом: не помещайте его в отчёты, Git, скриншоты и переписку. Для ранее созданной БД действуют уже сохранённые учётные данные.

4. Конфигурация и повседневный запуск

4.1. Файл .env

scripts/generate-local-env.py создаёт .env с правами 0600 и не печатает сгенерированные секреты. Существующий файл без флага --force не перезаписывается. Основные настраиваемые параметры:

Группа	Переменные	Назначение
PostgreSQL	POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_PORT	Локальная БД и порт
Приложение	BACKEND_PORT, FRONTEND_PORT, FRONTEND_ORIGINS	HTTP-доступ и same-origin
Bootstrap	BOOTSTRAP_ADMIN_EMAIL, BOOTSTRAP_ADMIN_PASSWORD	Первичный локальный ADMIN
Stack Exchange	STACK_EXCHANGE_SITE, STACK_EXCHANGE_TAG, STACKEXCHANGE_KEY	Источник и необязательный API key
Модели	EMBEDDING_MODEL, LLM_MODEL, RERANKER_MODEL	Локальные модели поиска/RAG
Индекс	QDRANT_ALIAS, AUTO_ENQUEUE_DOCUMENT_REINDEX	Активный индекс и постановка document reindex jobs

Секреты. Не коммитьте .env. Не публикуйте POSTGRES_PASSWORD, BOOTSTRAP_ADMIN_PASSWORD и STACKEXCHANGE_KEY. Файл .env.example содержит только примеры и placeholders.

4.2. Обычные команды

Команда	Действие
make demo-up	Запустить все восемь сервисов без удаления данных
make demo-status	Показать состояние и health контейнеров
make demo-logs	Поток последних 200 строк основных логов; выход — Ctrl+C
make demo-restart	Перезапустить прикладные сервисы
make demo-down	Остановить стек, сохранив persistent volumes
make demo-check	Сформировать acceptance-отчёт по доступным live-проверкам

5. Демонстрационное наполнение

На свежей БД `make first-run` уже выполняет bootstrap и явный demo seed. Bootstrap создаёт роли, permissions, системные настройки, источник «Stack Overflow на русском» и первичного ADMIN. Demo seed создаёт демонстрационных пользователей, документы, ответы, теги, чанки и примеры операционных данных.

Повторный идиempотентный demo seed при уже запущенной БД:

```
docker compose run --rm --no-deps \  
  -e SEED_DEMO_DATA=true backend \  
  python -m app.scripts.seed_demo
```

Разделяйте данные. Демо-документы предназначены для интерфейсной демонстрации и не считаются реальным корпусом Stack Overflow. Реальный корпус загружается одним из двух способов ниже.

6. Реальный корпус через Stack Exchange API

API-канал предназначен для небольшого smoke, контролируемого импорта и последующих incremental-обновлений. Worker обрабатывает pagination, quota, backoff, Retry-After, retry, checkpoint и cancellation; данные сохраняются в PostgreSQL.

6.1. Получение SOURCE_ID

Источник создаётся bootstrap. UUID можно безопасно прочитать из PostgreSQL без изменения данных:

```
docker compose exec -T postgres sh -lc \  
  'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" -Atc \  
  "SELECT id, name FROM sources ORDER BY created_at;"'
```

Скопируйте UUID строки «Stack Overflow на русском» и подставляйте вместо <uuid>. В ADMIN UI источник находится на странице <http://localhost:8080/admin/sources>; перед импортом используйте кнопку «Проверить» для малого сетевого запроса и проверки quota.

6.2. Безопасный smoke

```
make import-smoke SOURCE_ID=<uuid>
```

Фактический target вызывает INITIAL sync с ограничениями: не более 100 документов и двух страниц API. Задание помещается в durable очередь PostgreSQL и выполняется ingestion worker.

Наблюдение:

```
make worker-health  
make ingestion-report  
make worker-logs
```

В интерфейсе прогресс, checkpoint, jobs и события доступны ADMIN. Остановка через UI выставляет cancellation request; worker завершает безопасную точку и сохраняет checkpoint.

6.3. Ограниченный минимальный импорт

```
RUN_LIVE_MINIMUM_IMPORT=1 \  
make import-minimum SOURCE_ID=<uuid>
```

Команда явно разрешает живой импорт с пределами --max-documents 5000 и --max-pages 60. Проверьте quota и свободное место заранее.

6.4. Incremental-обновление

```
make sync-incremental SOURCE_ID=<uuid>
```

Make-target ограничен двумя страницами. Для штатного продолжения через интерфейс используйте режим AUTO: после завершённого initial snapshot система выбирает incremental sync; checkpoint предотвращает потерю уже зафиксированного прогресса.

6.5. Полный API-импорт — только осознанно

```
make backup  
  
RUN_FULL_IMPORT=1 \  
make import-full SOURCE_ID=<uuid>
```

Ресурсоёмкая операция. Команда не имеет лимита документов и защищена `RUN_FULL_IMPORT=1`. Учитывайте API quota, backoff и длительность. Не запускайте вторую синхронизацию того же источника параллельно. Не удаляйте worker во время активной транзакции; используйте штатную отмену в UI.

7. Массовое наполнение из официального Data Dump

Для первичного bulk-наполнения без расходования API quota реализован потоковый импорт официального XML Data Dump. Импортёр читает `Posts.xml` и `Users.xml` через 7z, выбирает вопросы с меткой `python`, присоединяет ответы и авторов, затем использует общий ingestion pipeline: очистку HTML, сохранение кода, hashing, дедупликацию, revisions, чанкинг и постановку переиндексации.

7.1. Скачать архив с возобновлением

```
mkdir -p artifacts/raw/stackexchange

curl -L --fail --retry 5 --continue-at - \
  --output artifacts/raw/stackexchange/ru.stackoverflow.com.7z \
  https://archive.org/download/stackexchange_20260331/stackexchange_20260331/ru.stackoverflow.com.7z
```

URL и snapshot приведены ровно в `docs/stackexchange-data-sources.md` текущего commit. Перед загрузкой проверьте доступное место командой `make disk-report`.

7.2. Запустить импорт

```
make backup

RUN_DATA_DUMP_IMPORT=1 \
DUMP_ARCHIVE=artifacts/raw/stackexchange/ru.stackoverflow.com.7z \
DUMP_TARGET_TOTAL=25000 \
make import-dump
```

Защитный флаг обязателен. По умолчанию Makefile передаёт target 25 000, concurrency 4 и лимит 25 failures. Архив монтируется в worker read-only; артефакты пишутся в каталог artifacts. Отчёт по умолчанию: `artifacts/raw/stackexchange/dump-import-report.json`.

7.3. Восстановить неполные ветки ответов

```
RUN_DATA_DUMP_IMPORT=1 \
DUMP_REPAIR_ANSWER_GAPS=1 \
DUMP_ARCHIVE=artifacts/raw/stackexchange/ru.stackoverflow.com.7z \
make import-dump
```

Repair-режим предназначен для импорта, прерванного старой версией или оставившего неполные ветки. Повторная обработка идемпотентна по (`source_id`,

external_id) и content hash; существующие неизменённые документы не должны дублироваться.

Практический выбор канала. Для большого первичного корпуса используйте официальный Data Dump. Для свежих изменений после snapshot — Stack Exchange API в AUTO/INCREMENTAL. Проект не использует обход quota, ротацию IP, CAPTCHA bypass или агрессивный HTML scraping.

8. Построение и проверка индекса

PostgreSQL остаётся источником истины. Indexer формирует sparse BM25 и dense embeddings, записывает points в новую Qdrant collection и переключает активный alias pyanswer_chunks_current только после успешного blue-green rebuild.

8.1. Убедиться, что модели доступны

```
make models-check
make ollama-status
make indexer-up
```

8.2. Полная переиндексация

```
CONFIRM_INDEX_FULL=YES make index-full
```

Ресурсоёмкая операция. index-full только ставит защищённое задание полного rebuild. Дождитесь его завершения и переключения ACTIVE alias. На CPU embedding и RAG существенно медленнее GPU.

8.3. Наблюдение и контроль согласованности

```
make index-status
make index-logs
make index-check
```

index-check сравнивает PostgreSQL chunks и Qdrant points. Failed rebuild не должен переключать alias. При расхождении сначала сохраните отчёт и выясните причину; не удаляйте PostgreSQL volume.

9. Финальная проверка наполнения

После завершения jobs выполните:

```
make demo-status
make models-check
make worker-health
make ingestion-report
make corpus-manifest
make data-check
make index-check
make disk-report
make demo-check
```

Проверка	Что подтверждает	Результат
demo-status	Контейнеры запущены, healthchecks проходят	Состояния Compose
ingestion-report	Число документов, ответов, чанков, revisions и failures	JSON в терминале
corpus-manifest	Состав фактического корпуса	Артефакт в artifacts/
data-check	Согласованность реляционных данных	PASS/WARNING/FAIL
index-check	Соответствие chunks и Qdrant points	PASS/WARNING/FAIL
demo-check	Доступные acceptance/live проверки	PASS/WARNING/FAIL/NOT_RUN

Откройте UI и вручную проверьте BM25, Vector и Hybrid Search, страницу документа, источники результата и RAG-ответ. Для RAG модель qwen3:8b должна быть загружена и доступна Ollama.

10. Типовые проблемы

Симптом	Проверка	Действие
Порт занят	<code>make demo-status</code> , системный список портов	Изменить <code>FRONTEND_PORT</code> , <code>BACKEND_PORT</code> , <code>POSTGRES_PORT</code> или <code>QDRANT_PORT</code> в локальном <code>.env</code>
Model missing	<code>make models-check</code>	<code>make models-pull</code> ; для reranker — <code>make reranker-model-pull</code>
Reranker unhealthy	<code>make reranker-logs</code>	Проверить наличие модели и объём RAM; базовая конфигурация использует CPU
API quota / backoff	ADMIN sources, job events, worker logs	Дождаться указанной задержки; checkpoint сохраняется. Для bulk использовать официальный dump
Job остановился	<code>make worker-health</code> , Jobs UI	Проверить lease/heartbeat/failure; использовать штатный retry или новый AUTO sync
Недостаточно RAM	Ollama/reranker logs	Уменьшить <code>LLM_NUM_CTX</code> , оставить reranker на CPU, <code>OLLAMA_NUM_PARALLEL=1</code>
Chunks $\neq$ points	<code>make index-check</code>	Сохранить отчёт, затем при необходимости выполнить защищённый full reindex
UI не открывается	<code>make demo-status</code> , <code>make demo-logs</code>	Проверить health backend/frontend и открыть <code>http://localhost:8080</code>

11. Остановка, бэкап и сохранность данных

Обычная остановка:

```
make demo-down
```

Compose volumes `postgres_data`, `qdrant_data`, `ollama_models` и `reranker_models` сохраняются. Следующий запуск — `make demo-up`.

Бэкап перед массовым импортом или переиндексацией:

```
make backup
```

Безопасная проверка восстановления в disposable-окружении:

```
make restore-check BACKUP_PATH=backups/<timestamp>
```

Не выполняйте без отдельного решения. Не удаляйте volumes, не восстанавливайте backup поверх основной БД и не используйте destructive reset. make demo-reset сам не удаляет production-like volumes и требует явного подтверждения.

12. Короткие готовые сценарии

Сценарий А — свежая локальная демонстрация

```
make first-run
make models-pull
make reranker-model-pull
make demo-up
make demo-status
# открыть http://localhost:8080
```

Сценарий В — небольшой реальный импорт через API

```
make demo-up
# получить SOURCE_ID по разделу 6.1
make import-smoke SOURCE_ID=<uuid>
make worker-health
make ingestion-report
make index-check
```

Сценарий С — первичное bulk-наполнение из dump

```
make demo-up
make backup
# скачать архив по разделу 7.1
RUN_DATA_DUMP_IMPORT=1 \
DUMP_ARCHIVE=artifacts/raw/stackexchange/ru.stackoverflow.com.7z \
DUMP_TARGET_TOTAL=25000 \
make import-dump
CONFIRM_INDEX_FULL=YES make index-full
make index-check
make demo-check
```

Контрольный лист готовности

- ☐ make demo-status показывает здоровые сервисы.
- ☐ Ollama содержит embedding- и generation-модели.
- ☐ Reranker запущен либо корректно отражён как optional/unavailable.

- ☐ В PostgreSQL присутствуют реальные документы, ответы и чанки.
- ☐ Активен Qdrant alias `pyanswer_chunks_current`.
- ☐ `index-check` не выявляет необъяснённого расхождения.
- ☐ В UI работают поиск, открытие документа и RAG с источниками.
- ☐ `.env` и локальные секреты не помещены в Git и отчётные материалы.

Итог. После выполнения выбранного сценария PostgreSQL содержит корпус и операционные данные, Qdrant — производный индекс, Ollama — локальные модели, а PyAnswer доступен через единый адрес `http://localhost:8080`.

Команды сверены с фактическим репозиторием на commit `60cc0680cba0934d2359701778e743f51f397a8d`.
Полный импорт, `model pull` и `full reindex` в рамках формирования документа не выполнялись.